DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

Student Name: Mohammed Affan M

Register Number: 192572169

Department: Computer Science and Engineering

Brute-Force and Exhaustive-Search Techniques

Includes explanations, pseudocode, Python implementations, sample test evidence, complexity analysis, and redesign validation.

Note on experimental evidence

The assignment asks for implementation and raw experimental measurements. The operation counts below are derived from the stated test cases and algorithm definitions, so they are reproducible by hand. Execution times are intentionally not invented because they depend on the computer and runtime. Run the included Python code on your own machine to record actual times and attach screenshots/raw output before submission.

All examples use Python 3 and are designed to be easy to modify. For a fair comparison, use the same input for all algorithms within each test.

Q1 – Sorting Under Different Data Conditions (6 Marks)

1. Prediction and algorithmic reasoning

Selection Sort: scans the unsorted portion to find the minimum on every pass. It performs about n(n–1)/2 comparisons regardless of input order. It uses few swaps (at most n–1), but does not benefit much from nearly sorted data.

Bubble Sort: compares adjacent values and swaps inverted neighbours. With an early-stop flag, an already sorted list takes n–1 comparisons and no swaps. Reverse or highly disordered input causes many swaps and quadratic work.

Insertion Sort: inserts each next element into the already sorted prefix. It is very efficient on already/nearly sorted data because few shifts are needed; its worst case is reverse order, with quadratic comparisons and movements.

Input condition	Selection Sort prediction	Bubble Sort prediction	Insertion Sort prediction
Already sorted	Quadratic comparisons; few/no swaps	One pass with early stop; no swaps	Linear comparisons; no shifts
Nearly sorted	Similar comparisons to other inputs	Few swaps; may finish quickly	Very few shifts; usually best
Random order	Quadratic comparisons	Quadratic comparisons/swaps on average	Quadratic average movement
Reverse order	Quadratic comparisons; at most n–1 swaps	Maximum adjacent swaps	Maximum shifts; quadratic

2. Designed datasets and operation-count evidence

Dataset A (nearly sorted): [10, 20, 30, 40, 35, 50]. Dataset B (reverse ordered): [6, 5, 4, 3, 2, 1]. Edge case: [] (empty input) and [7] (one item).

Dataset	Selection comparisons	Bubble comparisons (early stop)	Insertion key comparisons	Observed interpretation
A: nearly sorted, n=6	15	9	6	Insertion uses only a small number of shifts; bubble detects sorted order after one swap pass.
B: reverse, n=6	15	15	15	All three perform quadratic-scale comparisons; bubble swaps 15 times and insertion shifts 15 times.
Empty / one item	0	0	0	All return immediately; no sorting comparisons.

For Dataset A, insertion comparisons are 1 + 1 + 1 + 2 + 1 = 6 using the usual comparison of the key against prior elements; the precise count depends on whether the final failed comparison is counted. Bubble count assumes a standard early-stop implementation. Counts must be defined consistently when reporting results.

3. Python implementation

```
def selection_sort(a):
    a = a.copy(); comps = swaps = 0
    n = len(a)
    for i in range(n - 1):
        m = i
        for j in range(i + 1, n):
            comps += 1
            if a[j] < a[m]:
                m = j
        if m != i:
            a[i], a[m] = a[m], a[i]
            swaps += 1
    return a, comps, swaps

def bubble_sort(a):
    a = a.copy(); comps = swaps = 0
    n = len(a)
    for end in range(n - 1, 0, -1):
        changed = False
        for j in range(end):
            comps += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1; changed = True
        if not changed:
            break
    return a, comps, swaps

def insertion_sort(a):
    a = a.copy(); comps = shifts = 0
    for i in range(1, len(a)):
        key = a[i]; j = i - 1
        while j >= 0:
            comps += 1
            if a[j] > key:
                a[j + 1] = a[j]; shifts += 1; j -= 1
            else:
                break
        a[j + 1] = key
    return a, comps, shifts

tests = {
    "nearly sorted": [10, 20, 30, 40, 35, 50],
    "reverse": [6, 5, 4, 3, 2, 1],
    "edge": []
}
for name, data in tests.items():
    print(name, selection_sort(data), bubble_sort(data), insertion_sort(data))
```

4. Design decision

Recommend **Insertion Sort** for the stated workload because about 95% of orders are already in priority order and only small batches are added. It takes advantage of existing order and can insert each new order into the correct position with limited movement. If the whole dataset is repeatedly sorted and becomes large, a more scalable O(n log n) algorithm may be preferable. The recommendation is based on the described input condition, not a claim that insertion sort is always best.

Q2 – When Is Sequential Search Appropriate? (4 Marks)

Sequential Search examines records from the beginning until the requested Tool ID is found or the list ends. It requires no sorting or index, so it suits small, frequently changing temporary lists.

Experimental design and comparison counts

List size	Successful: first item	Successful: middle item	Successful: last item	Unsuccessful
n = 5	1	3	5	5
n = 10	1	5	10	10
n = 100	1	50	100	100

For a successful search at position k (1-based), comparisons = k. For an unsuccessful search, comparisons = n. If successful targets are equally likely at any position, average successful comparisons are (n+1)/2; unsuccessful searches always require n comparisons.

```
def sequential_search(records, target):
    comparisons = 0
    for i, record in enumerate(records):
        comparisons += 1
        if record["tool_id"] == target:
            return record, comparisons
    return None, comparisons

tools = [
    {"tool_id": "T-184", "technician": "TECH-12", "bay": "B1"},
    {"tool_id": "T-205", "technician": "TECH-07", "bay": "B3"}
]
print(sequential_search(tools, "T-184"))
print(sequential_search(tools, "T-999"))
```

Engineering recommendation

Use sequential search when the active list is small, searches are occasional, and records are inserted/deleted often enough that maintaining an index would add unnecessary complexity. Stop recommending it when the list becomes large and lookup requests are frequent, because the cost grows linearly with n per request. For 20 facilities with many active records, use a hash table indexed by Tool ID for expected O(1) lookup, or a database index for persistent multi-user records. Updates remain practical with a suitable index.

Q3 – Brute-Force String Matching (6 Marks)

The brute-force method aligns the pattern at each possible text position and compares characters from left to right. If a mismatch occurs, it shifts the pattern by one position. It stops at the first match or after every alignment has been tested.

Test	Text	Pattern	Prediction
Low work	SECURITYLOG	LOG	Few alignments; pattern matches at the end.
High work	AAAAAAAAAA	AAAAB	Many repeated A comparisons before each mismatch; no match.
Additional	ABABACAB	BAC	A match is found after several alignments.

Implementation and comparison evidence

```
def brute_force_match(text, pattern):
    comparisons = 0
    if pattern == "":
        return 0, comparisons
    for start in range(len(text) - len(pattern) + 1):
        j = 0
        while j < len(pattern):
            comparisons += 1
            if text[start + j] != pattern[j]:
                break
            j += 1
        if j == len(pattern):
            return start, comparisons
    return -1, comparisons

cases = [
    ("SECURITYLOG", "LOG"),
    ("AAAAAAAAAA", "AAAAB"),
    ("ABABACAB", "BAC")
]
for text, pattern in cases:
    print(text, pattern, brute_force_match(text, pattern))
```

For the high-work case, there are 6 possible alignments (text length 10, pattern length 5). Each alignment compares four A characters successfully and then mismatches B against A, giving 6 × 5 = 30 comparisons. This is a deliberately repetitive case. Actual counts for the other cases follow the code's first-match stopping rule.

The worst-case number of character comparisons is O((n–m+1)m), commonly written O(nm), where n is text length and m is pattern length. Repeated prefixes in the pattern and text cause the same characters to be checked again at neighbouring alignments. Input length alone is therefore insufficient to describe the work: character arrangement and the location of a match also matter. For large log streams or many signatures, an algorithm such as KMP or a multi-pattern matcher can reduce repeated comparisons.

Q4 – Brute-Force Closest Pair (6 Marks)

For every pair of distinct drones, calculate Euclidean distance and retain the pair with the smallest distance. To avoid duplicate work, only examine pairs (i, j) where j > i.

Distance between points A=(x■,y■) and B=(x■,y■) is d = √((x■–x■)² + (y■–y■)²).

The number of unique pairs is C(n,2) = n(n–1)/2. This follows because there are n choices for the first point and n–1 for the second, but each pair is counted twice in n(n–1), so divide by 2.

Number of drones n	Unique pairs / distance calculations	Growth
10	10×9/2 = 45	Small
100	100×99/2 = 4,950	110 times as many as n=10
1,000	1,000×999/2 = 499,500	About 101 times n=100

```
from math import hypot

def closest_pair(points):
    if len(points) < 2:
        return None, 0
    best_pair = None
    best_distance = float("inf")
    checks = 0
    for i in range(len(points) - 1):
        for j in range(i + 1, len(points)):
            checks += 1
            d = hypot(points[i][0] - points[j][0],
                      points[i][1] - points[j][1])
            if d < best_distance:
                best_distance = d
                best_pair = (points[i], points[j])
    return (best_pair, best_distance), checks

for n in (10, 100, 1000):
    points = [(i, (i * 37) % 101) for i in range(n)]
    result, calculations = closest_pair(points)
    print(n, calculations, result)
```

Scalability and redesign

For this assignment, define an acceptable response time as at most 1 second on the test computer. Record actual time with time.perf_counter() and report the computer/runtime used. The pair-count table predicts the trend, but the precise point at which the one-second limit is exceeded must be measured on the actual environment; it cannot be determined reliably from pair counts alone.

The bottleneck is examining every pair, which gives Θ(n²) distance calculations. A redesigned solution should sort points by coordinate and use geometric bounds to avoid comparing pairs that cannot improve the current minimum. A divide-and-conquer closest-pair algorithm can achieve O(n log n) time, at the cost of extra implementation complexity and memory.

Q5 – Brute-Force Convex Hull (7 Marks)

A convex hull is the smallest convex polygon containing every input point. For each pair of points A and B, treat the line AB as a possible boundary edge. Determine which side each remaining point lies on using the orientation cross product.

For A=(x■,y■), B=(x■,y■), and P=(x■,y■): cross = (x■–x■)(y■–y■) – (y■–y■)(x■–x■). Positive and negative values indicate opposite sides; zero means collinear.

Pseudocode

```
HULL(points):
    edges = empty set
    for each pair A, B with A != B:
        positive = false
        negative = false
        for each point P other than A and B:
            c = cross(A, B, P)
            if c > 0: positive = true
            if c < 0: negative = true
        if not (positive and negative):
            add edge (A, B) as a supporting-line candidate
    remove duplicate edges
    order accepted hull vertices around their centroid
    return ordered vertices
```

A candidate line is a supporting line when all other points are on one side or on the line; this means the line does not cut through the point set. For robust code, use a small tolerance for floating-point coordinates, handle collinear points explicitly, and remove duplicate points. When multiple points lie on one hull edge, retain the extreme endpoints for a minimal polygon.

Implementation

```
def cross(a, b, p):
    return ((b[0] - a[0]) * (p[1] - a[1])
            - (b[1] - a[1]) * (p[0] - a[0]))

def brute_hull(points):
    pts = list(dict.fromkeys(points))
    n = len(pts)
    if n <= 1:
        return pts
    edges = set()
    for i in range(n):
        for j in range(i + 1, n):
            pos = neg = False
            for k in range(n):
                if k == i or k == j:
                    continue
                c = cross(pts[i], pts[j], pts[k])
                if c > 0: pos = True
                elif c < 0: neg = True
            if not (pos and neg):
                edges.add((i, j))
    # Collect supporting-edge endpoints; order them around their centroid.
    ids = sorted({v for edge in edges for v in edge})
    candidates = [pts[i] for i in ids]
    cx = sum(p[0] for p in candidates) / len(candidates)
    cy = sum(p[1] for p in candidates) / len(candidates)
    from math import atan2
    return sorted(candidates, key=lambda p: atan2(p[1]-cy, p[0]-cx))

tests = [
    [(0,0),(4,0),(4,3),(0,3),(2,1)],  # interior point
    [(0,0),(2,1),(4,0),(3,3),(1,4),(2,2)] # irregular set
]
for pts in tests:
    print("Hull:", brute_hull(pts))
```

The first test has four corner points and one interior point (2,1); the interior point should not form part of the boundary. The second irregular configuration checks that the algorithm is not tailored only to rectangles. The returned vertices can be visualised by plotting them in returned order and joining the final vertex back to the first.

The three nested loops give $O(n^3)$ time in this straightforward implementation: $O(n^2)$ candidate pairs, each checked against $O(n)$ points. The geometric rule supports correctness because a line that has points strictly on both sides passes through the interior and cannot be a supporting hull edge. Collinear points and degenerate sets (all points collinear) require special handling; the simple version is intended for instructional test cases.

Q6 – Exhaustive Search and Combinatorial Growth (8 Marks)

Represent each package subset by an n-bit mask. Bit i is 1 if package i is selected and 0 otherwise. Enumerate every mask from 0 to 2■–1, sum its weights and values, reject subsets whose weight exceeds the payload limit, and keep the feasible subset with the greatest value.

Small instance and correctness check

Package	Weight	Humanitarian value
A	2	6
B	3	8
C	4	9
D	5	10

Payload limit = 7. The feasible subset A+B+C has weight 9 and is infeasible; A+B has weight 5 and value 14; A+C has weight 6 and value 15; B+C has weight 7 and value 17; D+A has weight 7 and value 16. The best feasible subset is B+C, with total weight 7 and value 17. Other combinations are checked by the exhaustive algorithm, including the empty set.

```
def exhaustive_packages(packages, capacity):
    # Each package is (ID, weight, value)
    n = len(packages)
    best_value = -1
    best_weight = 0
    best_subset = []
    candidates = 0
    for mask in range(1 << n):
        candidates += 1
        weight = value = 0
        subset = []
        for i, (name, w, v) in enumerate(packages):
            if mask & (1 << i):
                subset.append(name)
                weight += w
                value += v
        if weight <= capacity and value > best_value:
            best_value, best_weight, best_subset = value, weight, subset
    return best_subset, best_weight, best_value, candidates

packages = [("A",2,6), ("B",3,8), ("C",4,9), ("D",5,10)]
print(exhaustive_packages(packages, 7))
```

Growth prediction and evidence

Packages n	Subsets examined = 2■	Increase from previous row
5	32	—
10	1,024	32× from n=5
15	32,768	32× from n=10
20	1,048,576	32× from n=15

Every extra package doubles the number of subsets: 2■■¹ = 2·2■. Consequently, adding five packages multiplies the search space by 32. The algorithm is O(n·2■) because each of the 2■ subsets may inspect up to n packages; memory use for the current subset is O(n), excluding input.

For a deployment rule, choose a response-time limit of 1 second for the planning prototype. Measure n = 5, 10, 15, and 20 with time.perf_counter() on the target computer. The largest deployable n is the largest measured size that stays under the limit with acceptable reliability. The subset count shows why exhaustive search is suitable for small package sets but may become impractical as n grows; the exact cutoff depends on hardware and implementation.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected limitation: exhaustive package selection from Q6. This is a strong example because the number of candidate solutions doubles whenever one package is added, making growth measurable and easy to compare.

A. Diagnose the bottleneck (2 marks)

The main cost is repeated evaluation of every possible subset. There are 2■ subsets and up to n package checks per subset, so work grows as O(n·2■). At n=20, 1,048,576 subsets must be checked; at n=25, 33,554,432 subsets are checked, which is 32 times as many as n=20. The original algorithm repeats weight/value summation for overlapping subsets.

B. Design objective (2 marks)

For integer package weights and a moderate capacity, reduce repeated subset enumeration by reusing the best value already computed for smaller capacities. Preserve the optimal value and selected package list while aiming for O(nW) time, where W is the integer payload capacity.

C. Revised strategy: dynamic programming (4 marks)

Use a table dp[i][w] to store the maximum value achievable using the first i packages with capacity w. For package i with weight wi and value vi, either exclude it or include it if wi ≤ w. The recurrence is dp[i][w] = max(dp[i–1][w], vi + dp[i–1][w–wi]) when wi ≤ w; otherwise dp[i][w] = dp[i–1][w].

```
DP_PACKAGES(packages, capacity):
    dp[0..n][0..capacity] = 0
    for i from 1 to n:
        (weight, value) = packages[i]
        for w from 0 to capacity:
            dp[i][w] = dp[i-1][w]
            if weight <= w:
                dp[i][w] = max(dp[i][w],
                               value + dp[i-1][w-weight])
    return dp[n][capacity]
```

This design reuses subproblem results instead of recomputing all subsets. Time is O(nW) and table memory is O(nW); a one-dimensional table can reduce memory to O(W), updating capacities in descending order. The trade-off is that the running time depends on numeric capacity W, so it may be poor if W is extremely large or weights are non-integer without scaling.

D. Prediction before implementation (1 mark)

Prediction: for moderate integer capacities, dynamic programming should evaluate far fewer states than 2■ subset enumeration as n increases. Compare elapsed time and the number of DP states (n×(W+1)) against exhaustive candidate count (2■).

E. Implement and validate (2 marks)

```
def dp_packages(packages, capacity):
    dp = [0] * (capacity + 1)
    for name, weight, value in packages:
        for w in range(capacity, weight - 1, -1):
            dp[w] = max(dp[w], value + dp[w - weight])
    return dp[capacity]

packages = [("A",2,6), ("B",3,8), ("C",4,9), ("D",5,10)]
print("Exhaustive optimum:", exhaustive_packages(packages, 7)[:3])
print("DP optimum value:", dp_packages(packages, 7))
```

For the small instance with capacity 7, both methods should return optimal value 17. The exhaustive version also identifies B+C; the compact one-dimensional DP code returns the value only. To validate the chosen subset, store predecessor decisions or use the full two-dimensional table and backtrack.

F. Defend the final design (2 marks)

The redesign achieves its objective when it returns the same optimum while reducing evaluated states. Exhaustive search remains useful for very small n, for validating other methods, and when capacity-based dynamic programming is unsuitable. Dynamic programming adds table-management complexity and is pseudo-polynomial: O(nW) is not polynomial in the number of bits needed to represent a very large W. The correct engineering choice depends on both n and W.

Overall Conclusion

The experiments and analysis show that algorithm choice should depend on input structure, dataset size, request frequency, and acceptable response time. Insertion Sort fits nearly sorted batches; Sequential Search fits small, frequently changing lists with low lookup demand; brute-force string matching, closest-pair, convex-hull, and

exhaustive subset search illustrate how repeated comparisons or candidate enumeration can become expensive. Operation counts and mathematical growth provide more reliable evidence than a single execution-time result. All actual timings should be measured on the submission computer and included with raw outputs.

Submission checklist: include source code, raw operation-count data, actual execution-time measurements and environment details, and plots/visualisations requested by the assignment.